A Customer journey to RHEL image mode

Hardening at scale without slowing delivery

Tihomir Hadzhiev
Senior specialist Solution
Architecture

Red Hat

At 2 a.m. a critical CVE drops

The pain isn’t patch speed – it’s the ability to prove

- A Nordic Tier-1 universal bank, mature Linux ops team
- Critical CVE drops overnight – 600 build agents with different drift histories
- Tooling shows what's installed now – not what ran at 23:58 incident…
- RHEL image mode makes the deployed artefact provable

This is thing that DORA, PSD2/PSD3 and NIS2 actually ask for

Regulations prescribe properties, not artefacts

What DORA requires

- Change traceability – auditable, attributed
- Recoverability – defined and tested RTOs
- Software composition assurance – tamper-evident
- PSD2/PSD3 + NIS2 layer on top: incident reporting, supply-chain

How image mode deliver, structurally

- Git commit + signed digest = attributed change record
- bootc rollback = tested recovery path, minutes not hours
- cosign-signed OCI digest = tamper-evident artefact
- Compliance as side-effect of delivery – not separate workstream

Red Hat

“I want it to work like my smartphone”

Factors that drove the search for a new solution

- Pain over regulatory process of CVEs
- Smaller footprint | decreased surface area of attack
- Enhanced security | hardened platform
- Quick turn around | lower downtime
- Easy rollbacks

FSI early adopter
Image mode for Red Hat Enterprise Linux

Red Hat

The environment

Pilot scope

Red Hat

Why this fleet

High velocity, single owner, non-customer-facing

- Developer-platform fleet: ~600 hosts – build agents, CI runners, back-office
- Selection: single owner, high patch velocity, reboot-tolerant, no customer traffic
- Not core banking – not the place to learn image mode's operating…
- Failure here delays a build pipeline – recoverable; failure in payments isn't

Choose pilot fleet by velocity and ownership, not by size

Pilot scope

600 hosts, 6 months, three gating criteria

- Gate 1: bootc rollback succeeds on 10+ incidents, no manual intervention
- Gate 2: audit-pass on one DORA quarter using image-digest provenance
- Gate 3: zero incidents attributable to the image delivery model itself

All met

What didn’t change

What DORA requires

- Net new headcount: zero — no new role, no new budget
- Red Hat Satellite still the management plane
- Insights / Lightspeed: image mode hosts register identically
- Image Builder still produces qcow2/AMI/vmdk/ISO from bootc

How image mode deliver, structurally

- Git commit + signed digest = attributed change record
- bootc rollback = tested recovery path, minutes not hours
- cosign-signed OCI digest = tamper-evident artefact
- Compliance as side-effect of delivery — not separate workstream

One RHEL, two modes

	Package mode	Image mode
Image creation	Image builder	Container tools
Updates	Packages (`dnf`)	Images (`bootc`)
Update distribution	rpm repository	Container registry
Management	Red Hat Insights, Satellite, Ansible	
Deployment footprint	Bare metal, VM, cloud, edge	

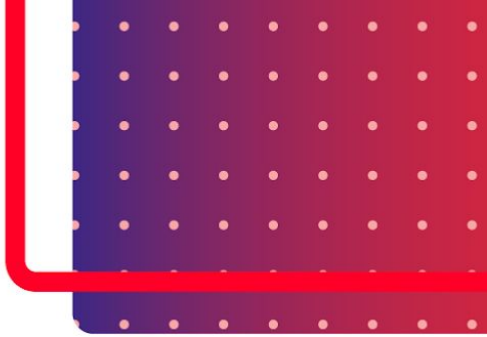
Why this fleet

High velocity, single owner, non-customer-facing

- Developer-platform fleet: ~600 hosts – build agents, CI runners, back-office
- Selection: single owner, high patch velocity, reboot-tolerant, no customer traffic
- Not core banking – not the place to learn image mode's operating…
- Failure here delays a build pipeline – recoverable; failure in payments isn't

Choose pilot fleet by velocity and ownership, not by size

Architecting the image



Red Hat

The full chain

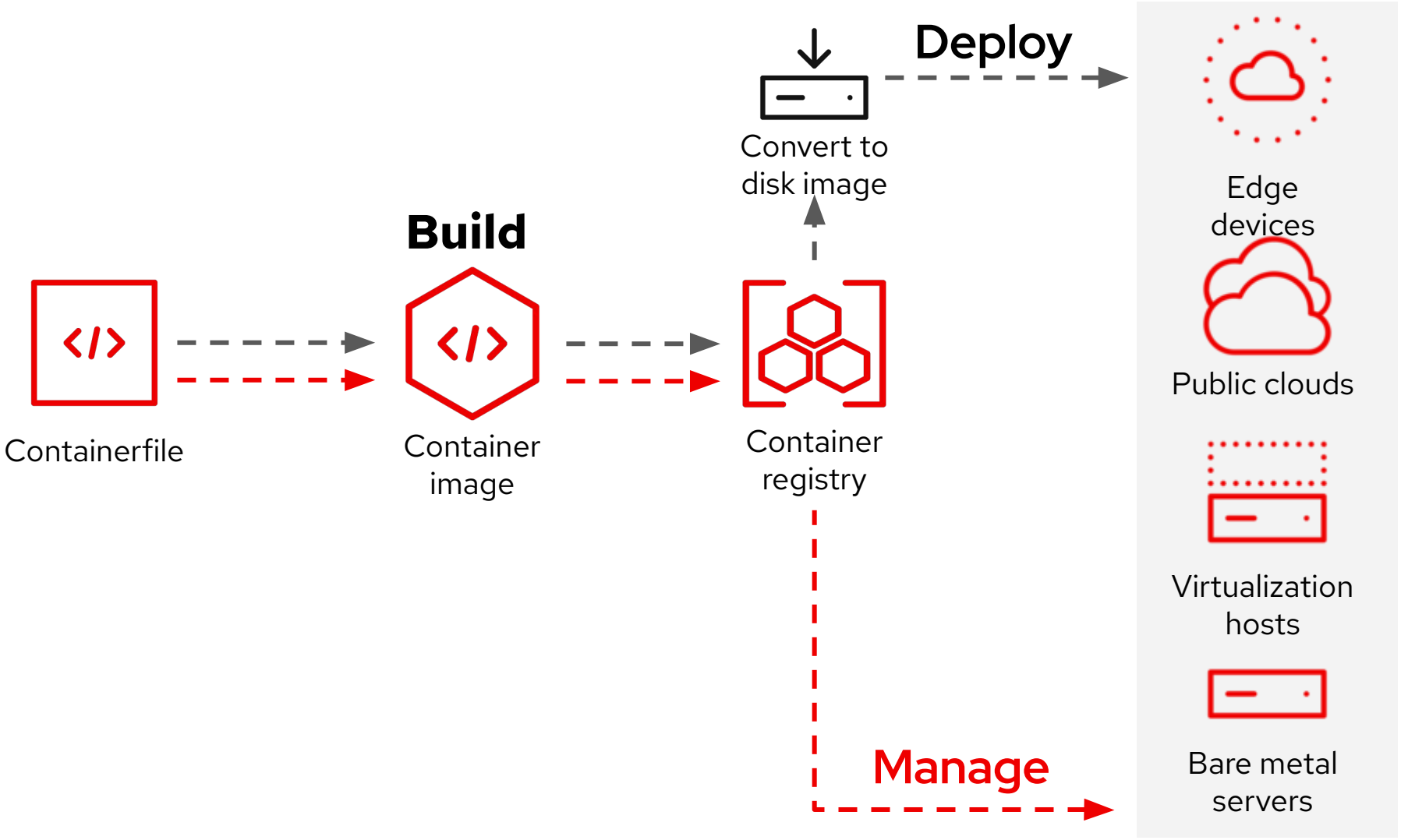
git commit → signed image → Satellite CV → bootc switch

- Git commit triggers CI/CD – same OCI build interface across pipelines
- podman build + cosign sign – every build git-tagged, cryptographically bound
- Quay receives signed image: tag history, layer diff, Clair scanning
- Satellite CV promotion: Dev → Test → Prod – explicit gate to...
- bootc switch + bootc status: digest, staged, rollback – your DORA record

Every OS change is a signed git-tagget OCI image. The management chan → Unmodified

Image mode for Red Hat Enterprise Linux

Simple. Consistent. Anywhere.



Red Hat

Layer 1

The base: every agent, every service, version-controlled

- Three layers: base (OS + agents) → security-overlay → app-overlay
- FROM line + git commit hash = complete description of the OS
- RHEL 10 base: 439 RPMs, ~785M compressed, ~2.2G on disk
- Satellite URL scopes pull to a specific Content View + lifecycle env
- No runbook, no wiki – the git log is the change log

```
FROM [satellite-url]/rhel-bootc:latest
RUN dnf install -y [system agents] [monitoring] && dnf clean all
COPY [site-wide config files] /etc/
RUN systemctl enable [standard services]
```

The base Containerfile is the authoritative description of the OS

Layer 2 + 3

Security inherits once, every app-overlay gets it free !

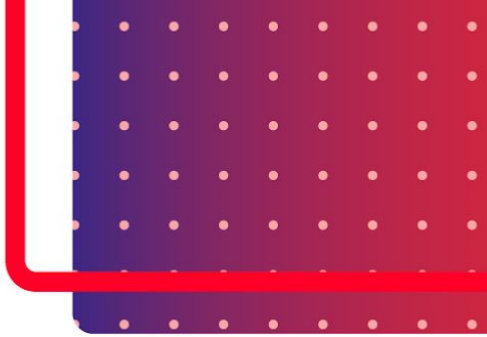
- Security-overlay derives from base – OpenSCAP hardening baked at build time
- oscap-im scans the image at build – finding lives in the registry

```
FROM [satellite-url]/corp-base-soe:latest
COPY tailoring.xml /usr/share/
RUN dnf install -y openscap-utils scap-security-guide && dnf clean all
RUN useradd -G wheel -p "<hashed>" admin
RUN oscap-im --profile pci-dss --results-arf /arf.xml \
    /usr/share/xml/scap/ssg/content/ssg-rhel10-ds.xml
```

- App-overlay derives from hardened SOE – workload packages only
- App teams don't write OpenSCAP policy – they inherit it
- Friction: per-host config (hostname, IP) moves to cloud-init – 3-4 weeks

```
FROM [satellite-url]/corp-hardened-soe:latest
RUN dnf install -y [build-agent-packages] && dnf clean all
COPY [build-agent-config files] /etc/
RUN systemctl enable [build-agent-services]
```

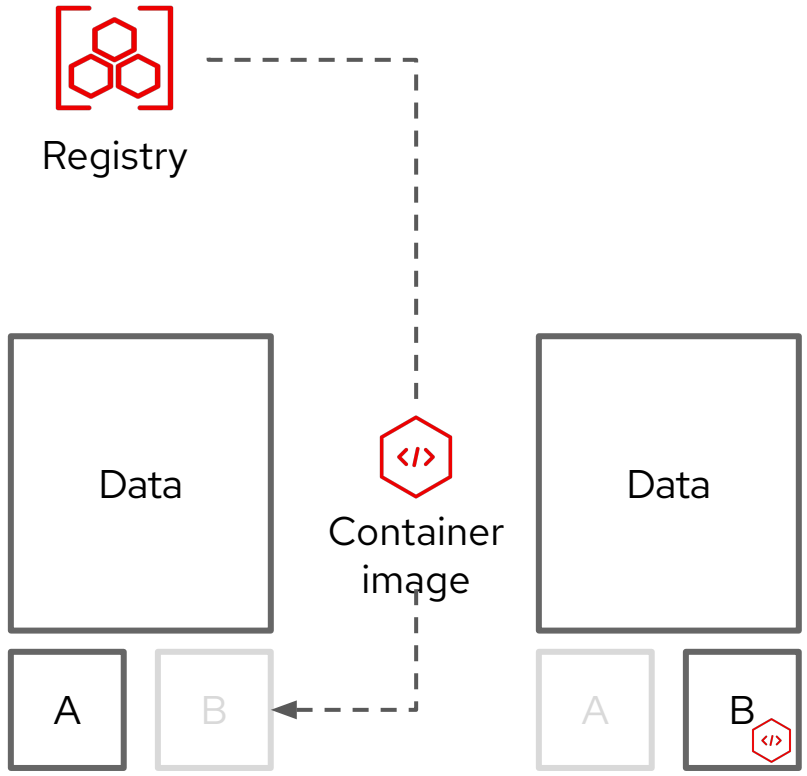
PIlot Results



Red Hat

Bootc: Image-based updates perfected

Immutable by default - secure by design



Transactional updates (A → B model)

Bootc uses composefs and ostree to convert the container image into the root filesystem on the host..

Roll forward or backwards

Updates are staged in the background and applied when the system reboots. The transactional model enables rollbacks for additional assurance

Upgrades have never been easier

While there are some limits, bootc enables moving between minor releases (i.e. RHEL 9.6 → 9.7), as well as major releases (i.e. RHEL 9 → 10)

Red Hat

Patch cadence - 6 weeks → 3 days

Because the model changed, no the tooling

- Before image mode: patch cycle on the developer-platform fleet ran approximately every 6 weeks.
- After image mode (pilot fleet, 600 hosts): ~3 days from CVE announcement
- The constraint moved from change-window scheduling to CI pipeline runtime. [unsourced — verify before stage]
- The counter-intuitive point: the process improvement came from the model change.

Automating a fundamentally slow process gives you a faster slow process.

Audit preparation - 4 weeks → 9 days

The evidence existed before the auditor asked

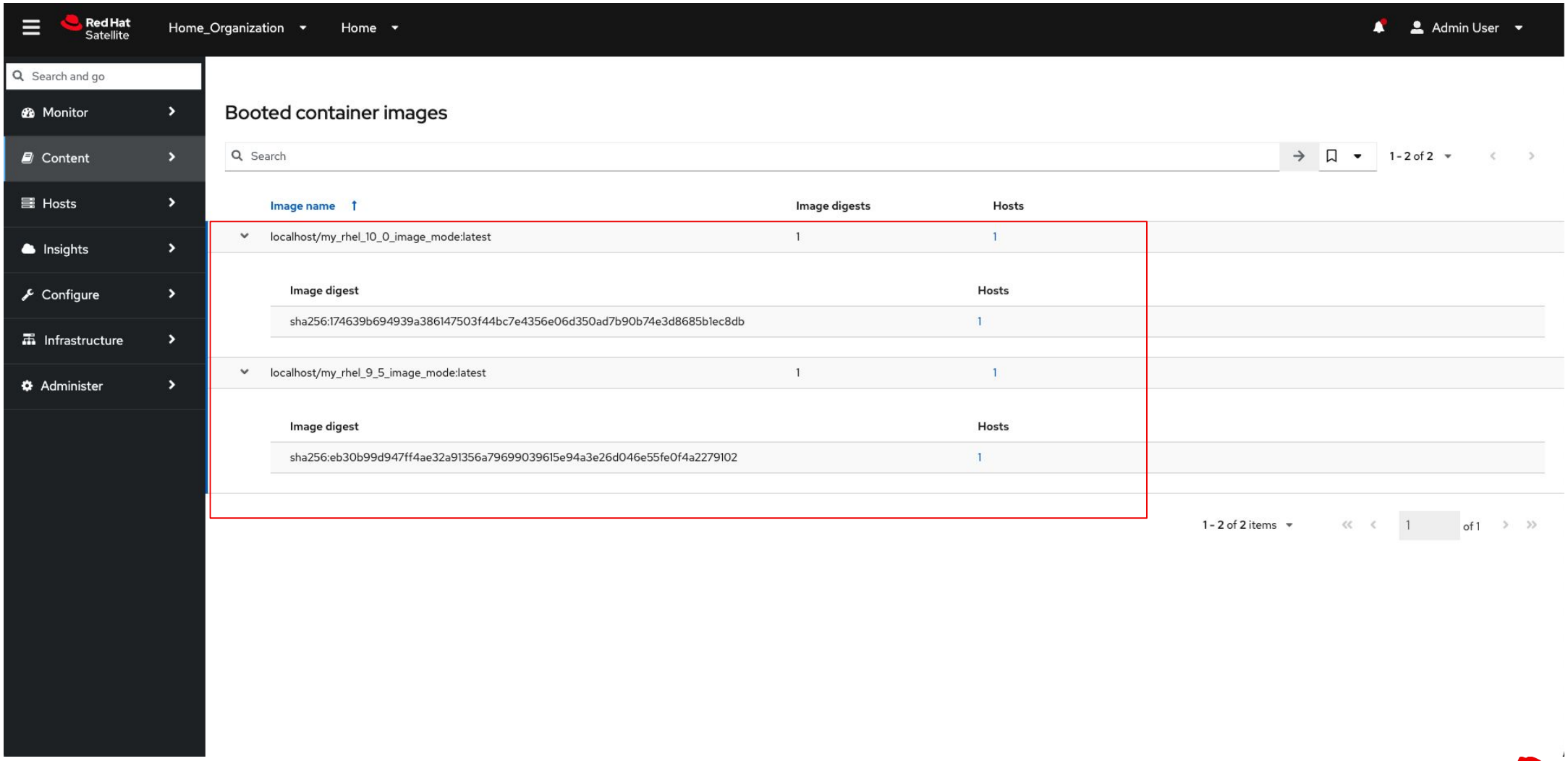
- Before: audit preparation involved gathering package lists, errata history, and system facts per host.
- After: ~9 days. The evidence is the registry.
- The Satellite shows current image digest per host. bootc status on any host returns: current image, staged image, rollback target.
- For DORA specifically: the audit evidence is a side-effect of the delivery model. The signed image digest in the registry was created at build time, before deployment.
- The friction: the 9-day result depends on auditors accepting image-digest provenance

Evidence is baked into the delivery model - not reconstructed!

Red Hat

Images currently running

Ok, so what images are currently in use?



Red Hat

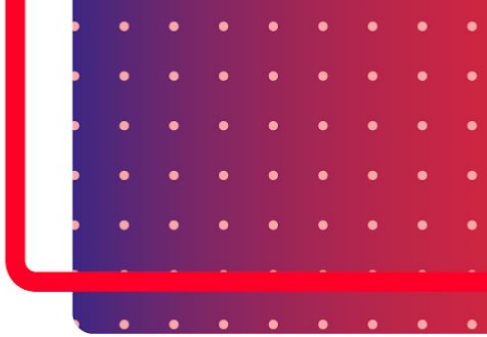
Rollback - 3 hours of archaeology → 11 minutes

Back to cryptographically verified artefact

- All rollbacks in the pilot: 100% executed as bootc rollback
- Time to full rollback: ~11 minutes on the pilot fleet (time from bootc rollback command to host confirmed running prior image and passing healthcheck).
- How it works: bootc's A/B model. The previous image is retained on the host in a second ostree deployment slot. bootc rollback switches the active slot. On reboot, the host is running the prior image.
- Every rollback is also a complete audit event
- Friction: the A/B model retains exactly one prior image.

Back to cryptographically verified artefact - not a package archaeology

The runbook



Red Hat

Git commit → rollback in one sequence

The pilot loop

- git commit -am "pin httpd to 2.4.62" –
- Pipeline: podman build -t quay.io/[org]/rhel-bootc-dev-agent:v26-$(date +%Y%m%d-%H%M) --sign-by-sigstore-private-key my_key.key
- Satellite Content View refresh – new image digest appears in CV; promote Dev → Test (show CV version number increment)
- On pilot host: bootc switch quay.io/[org]/rhel-bootc-dev-agent:prod – image staged in background
- bootc status – shows current image (old), staged image (new), rollback target (old)
- Reboot: systemctl reboot – host comes back on new image; bootc status confirms active image digest matches the cosign-signed digest from step 3
- Friction: Satellite needs to sync already signed images

Continuity

Red Hat

Satellite 6.18+

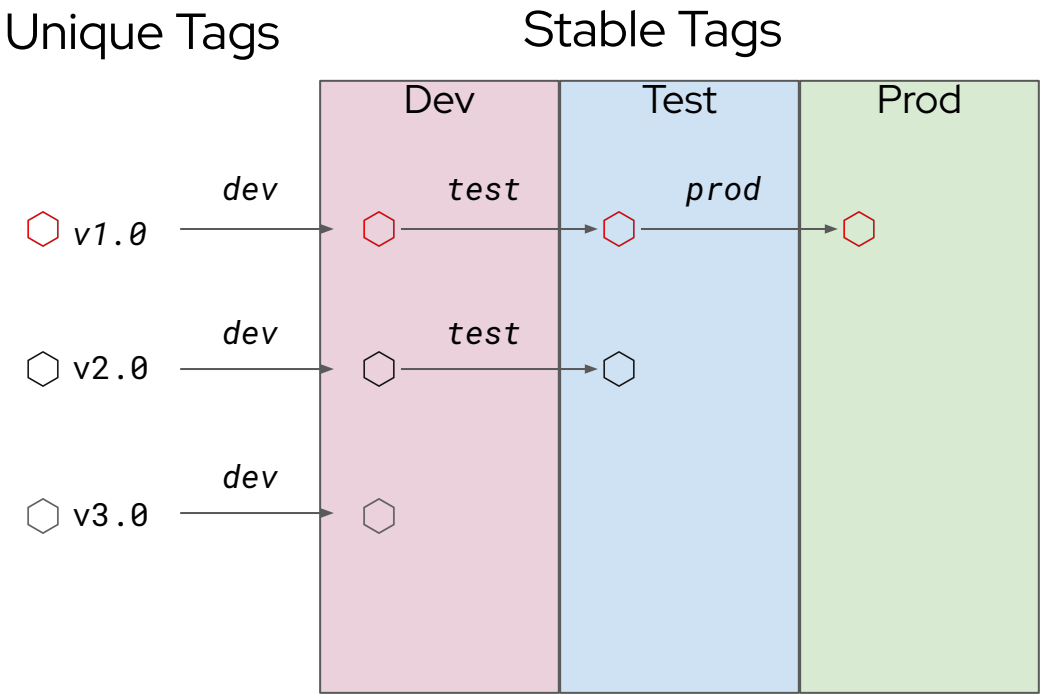
Satellite manages image mode hosts in the ame console

- 6.17 introduced image mode host support – unified All Hosts view
- 6.18 Vulnerability + Advisor powered by Lightspeed; offline CLA
- Pilot uses Satellite for inventory, CVs, remote exec, security compliance
- 6.19 Transient: emergency hotfixes that don't persist reboot

The governance model is the same - the artefact type id different

OS Updates via Container Registries

Tagging is powerful to version and promote updates



Tags offer simple versioning and visibility

Tags are simple to automate and use for promotions. Bootc will default to updating from a `repository:tag`.

Control updates via tagging

Combine tagging with the optional automatic updates to control fleets of systems via registry tags.

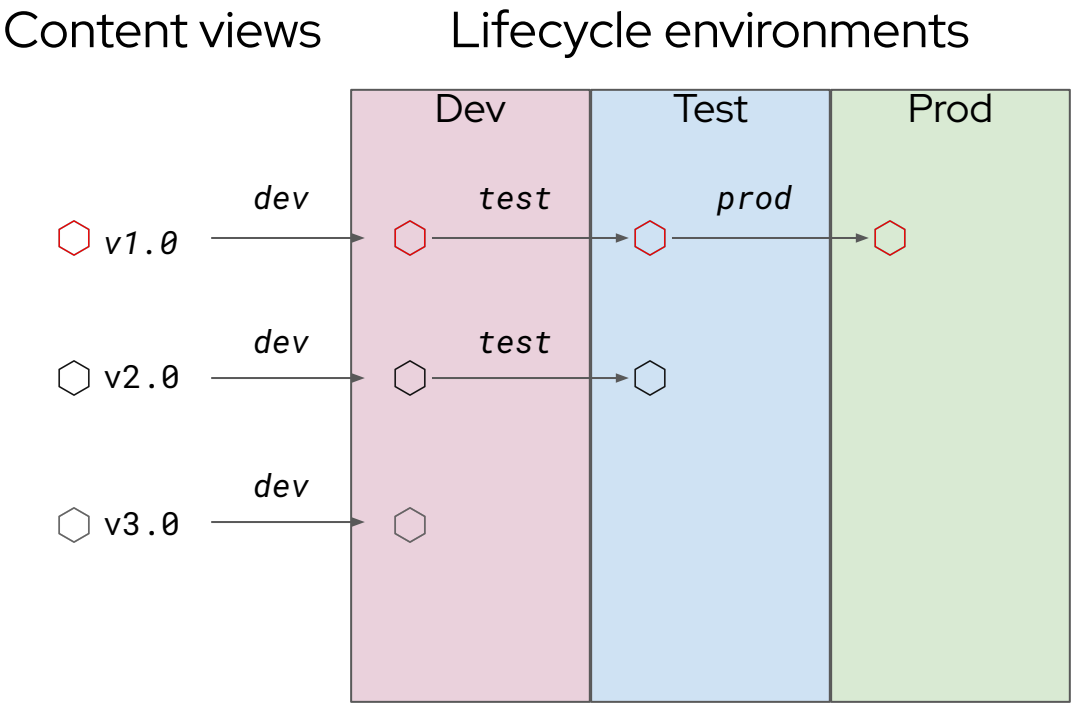
Standardized & scaleable infra

Container registries scale very well and any standard registry can be used.

Red Hat

OS Updates via Content views

Content views help promoting updates to OS in different lifecycle environments



Content Views offer controlled versioning and visibility

Content Views provide explicit versioning of OS content. Each published version is immutable, auditable, and clearly visible across environments.

Control updates via promotion

Updates are introduced once, validated in Dev and Test, and promoted to Prod only when approved. This enables predictable, repeatable OS updates across fleets.

Standardized & scalable lifecycle management

Satellite centralizes OS content, policies, and lifecycle workflows, scaling to thousands of systems while enforcing consistent update behavior.

Red Hat

The SOE update workflow

Satellite manages image mode hosts in the ame console

- Errata drops: update build CV, podman build, push signed image, tag
- Satellite remote exec runs bootc upgrade against target host group
- Hosts in lifecycle env pull on bootc upgrade – same governance model
- Pilot: 600 hosts in batches of 60, healthchecks between – ~1 hour
- All Hosts view shows running image digest – replaces inventory scripts

The Satellite based update workflow for image mode is the same pattern as package mode

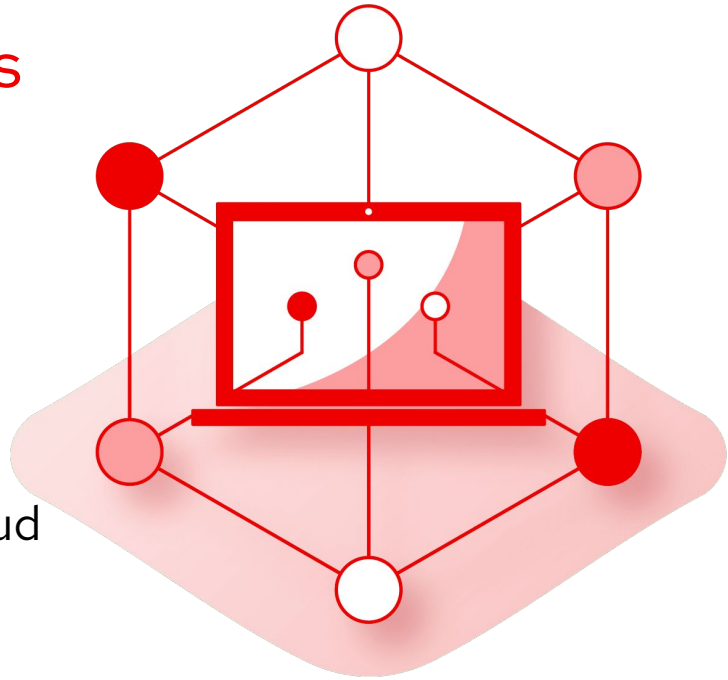
With the speed of light

Red Hat

Lightspeed on-prem

Vulnerability triage, no data egress, one image fixes the fleet

- 6.18 Vulnerability service: which hosts, which digest, fix available?
- Vulnerability service – CVE tracking without cloud connectivity
- Applicable = CVE affects running image; installable = fix already staged
- Pilot: on-prem was the only path – data sovereignty ruled out cloud
- AI-assisted triage now available where it previously wasn't



CVE triage with Lightspeed in Satellite 6.18 works on-prem, zero new data flows – and on image mode hosts, "remediation" is a new image push, not per-host package surgery

SCREENSHOT OF VULNERABILITY SERVICE

Lessons learned

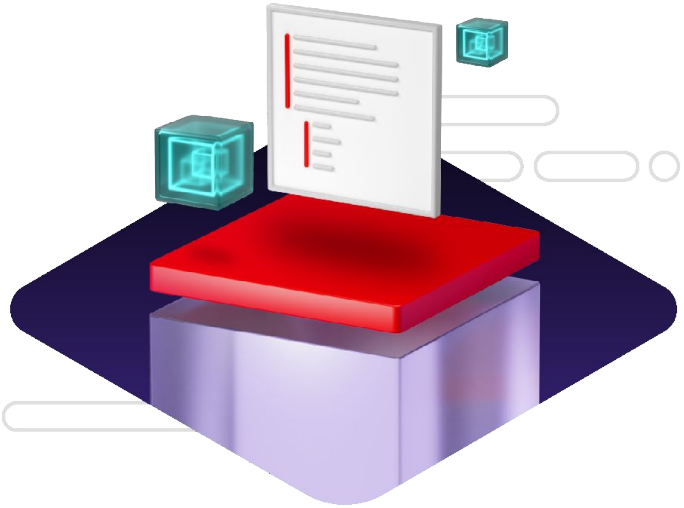
A way to go

Red Hat

What we learned

The manageable friction

- Containerfile refactor from Kickstart: 3-4 weeks calendar time
- Host-specific config boundary – moves to cloud-init, takes longer than expected
- 6.18 CV-for-bootc maturity – Quay tag workaround works, two steps not one
- Auditor education – first cycle needs a walkthrough, plan it before audit
- Not every workload is ready – one stayed on package mode, correctly

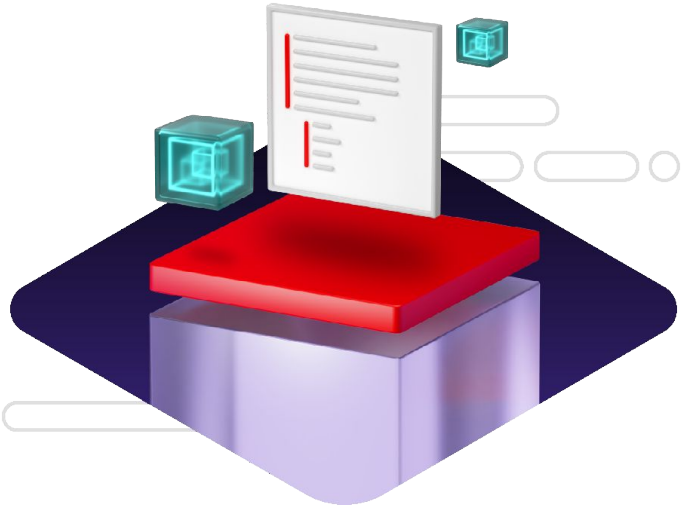


Time is needed to embrace the change

Starter playbook

The possible way to go

- Week 1: pick 20-50 non-prod hosts – single owner, high patch velocity
- Week 2: write base Containerfile, podman build locally, bootc status on VM
- Weeks 3-4: wire CI + signing, push to Quay, test bootc...
- Month 2: run one patching cycle via image build; test bootc rollback
- Month 3: engage compliance – walk them through digest chain as evidence

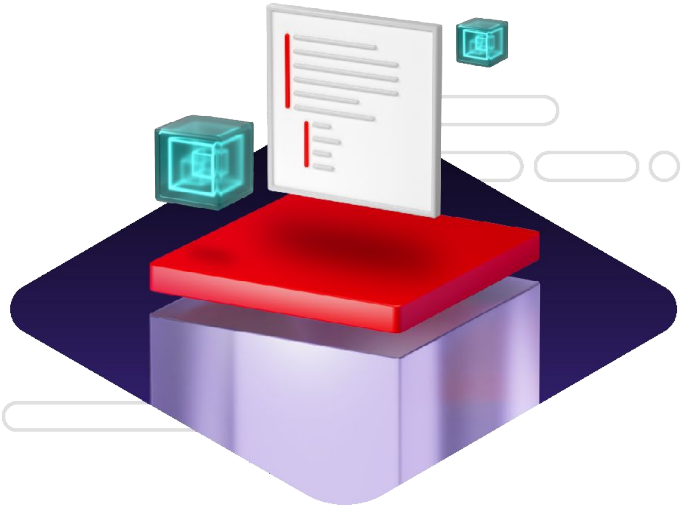


Five steps, five weeks, one team, zero production risk – the playbook is low-stakes at pilot scale.

What to do next week

Three paths – one entry point

- Evaluating a pilot? Pull rhel10/rhel-bootc:10, podman build – today
- Have Satellite? Check version. 6.18 for on-prem
- Regulated environment? Bring compliance to a session before next audit
- Other Summit sessions: BO1131 (Goldman Sachs), LT1461, LB1208



One concrete next step: pull the base image, write one Containerfile, run one bootc build

Thank you

linkedin.com/company/red-hat

facebook.com/redhat

instagram.com/red_hat

x.com/RedHatSummit

Red Hat

The Red Hat Image Mode team wants to hear from *you!*

Help shape what comes next.

Scan the QR code or go to:
red.ht/userfeedback

Share your pain points, experiences, and ideas.
Visit the UX team at the RHEL booth

Red Hat